Workflow and Tech Stack

databricks-agentic-de - a real-time healthcare data platform on Databricks where the platform's agents do the work and this project builds the glue and guardrails around them

Status as of 15 Oct 2026. Repository: github.com/sumanthreddy369/databricks-agentic-de

In one paragraph

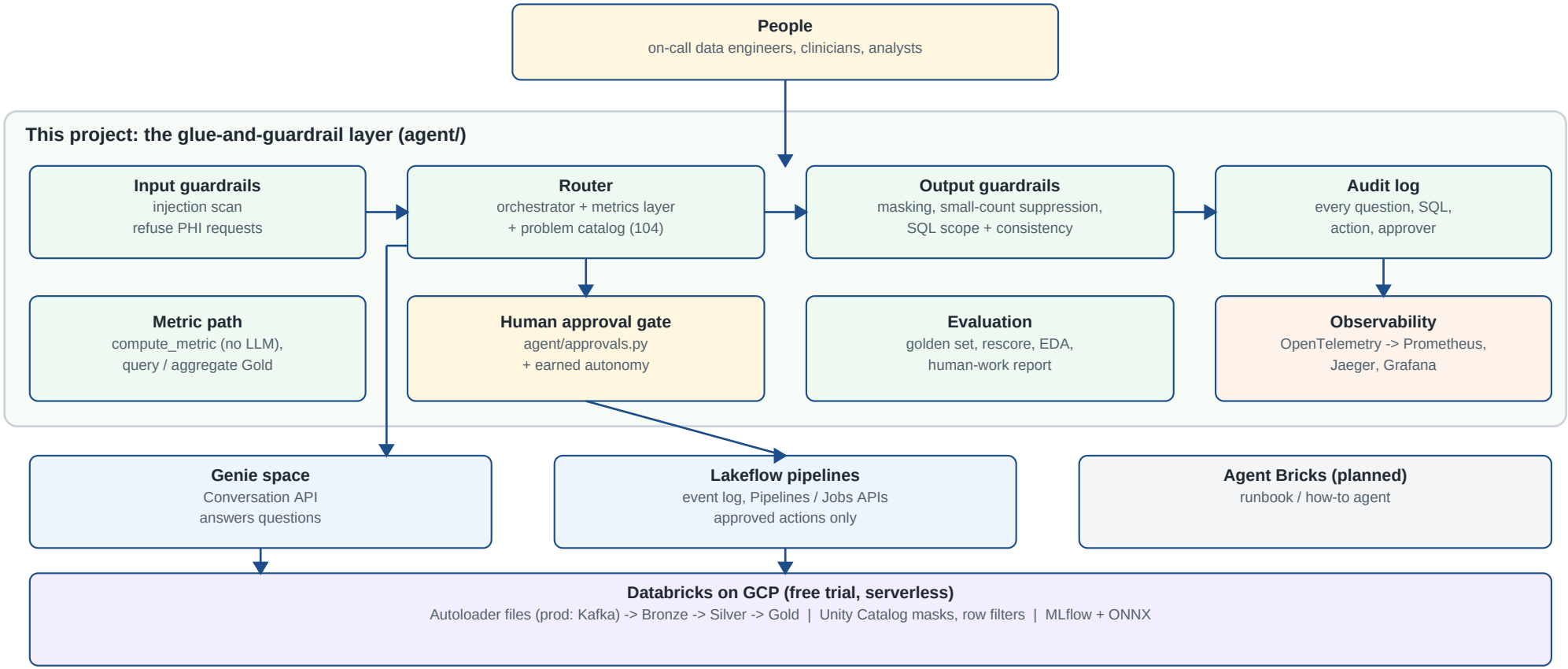
Patient events flow from a simulator into Databricks, through Bronze, Silver and Gold tables governed by Unity Catalog. Databricks' own agents do the data work: Genie answers analysts' questions and the pipeline APIs act on pipelines. This project is the layer around them that companies build themselves: it routes requests, checks everything going in and coming out, holds every change for a named human until that kind of change has earned autonomy, records everything, and measures it - answer quality, safety, and how much work still needed a person. The goal is to cut routine data-engineering and analyst work as fast as is safe.

Status at a glance

Area	State
Pipelines on Databricks (Free Edition; earlier GCP trial)	Runs - 103,359 simulated events (v5): 306 impossible vitals dropped; four identity checks catch 78/78 planted cases; a steward review queue decides
Unity Catalog masks	Runs - 1,388 of 1,388 names and MRNs masked in Gold; row filter and grants not tested
Genie behind the guardrails	Measured - 27-question golden set, 3 live runs; run 3 (Free Edition): 21/21 answers, 6/6 safety probes
Metrics layer	Runs - one definition per metric for Genie and compute_metric
Human approval, earned autonomy, human-work report	Runs - a live restart on Databricks ran only after a named approval
Observability (OTel, Prometheus, Jaeger, Grafana)	Runs locally on demo traffic through the real orchestrator
Data and deterioration detection	Measured - EDA done; with realistic physiology, model v2 + NEWS2 + rules catch 66% at 3.8% false alarms (simulated)
Not deployed	Terraform and Cloud Run (in the repo, never applied); the unit row filter is deployed but untested with workspace groups

1. Architecture

Three tiers: people at the top, this project's layer in the middle, Databricks' agents and platform underneath. Every request passes the input guardrails, every result the output guardrails, every action the approval gate; everything is audited, traced and measured.

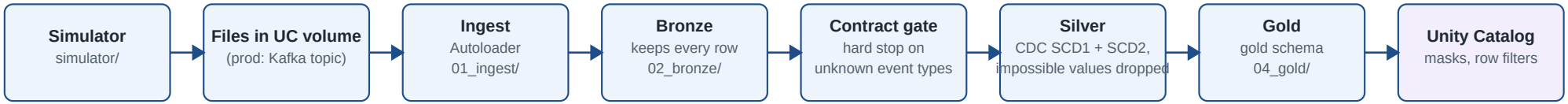


- **Platform does the work:** Genie (questions), Lakeflow pipelines and Jobs APIs (actions), Agent Bricks planned (runbooks).
- **We build glue + guardrails:** routing, the Genie gateway, the metrics layer, guardrails, approvals and earned autonomy, audit, evaluation, observability.
- **Metric path = baseline:** compute_metric answers routine metrics with no LLM in about 1.3 s; Genie (about 13 s) is measured against it.

2. Workflows

W1. Data flow: source to Gold

How a vital-sign reading becomes a governed Gold row. The free trial uses files in a Unity Catalog volume instead of Kafka; both feed identical columns to Bronze.



Step	What happens	Rules	Code	Status
Generate	10 min of events for 1,388 patients on a simulated clock	One file per simulated minute	simulator/file_feed.py	Runs - 103,301 events
Ingest + Bronze	Autoloader reads JSON lines; flatten, add ingestion time	Never drops a row	pipeline/01_ingest, 02_bronze	Runs on Databricks - raw = Bronze = 103,301
Silver	Contract gate, encounters (current + history), patients, vitals	Unknown event type fails the pipeline; 323 impossible vitals dropped	pipeline/03_silver/	Runs on Databricks - 99,213 vitals
Gold	dim_patients, fct_encounters, fct_encounter_history, fct_vitals, dim_providers, live vitals	Readings attributed to the unit at reading time	pipeline/04_gold/	Runs on Databricks
Govern	Masks on full_name / mrn declared in the pipeline; unit row filter	Agent never sees raw identifiers	pipeline/04_gold/, governance/	Partial - masks verified; row filter untested

- EDA of the same 103,301 events (docs/eda/): no duplicates, late events or broken lifecycles; 13 patient_ids shared by two people (random id collisions) - now flagged in Silver.

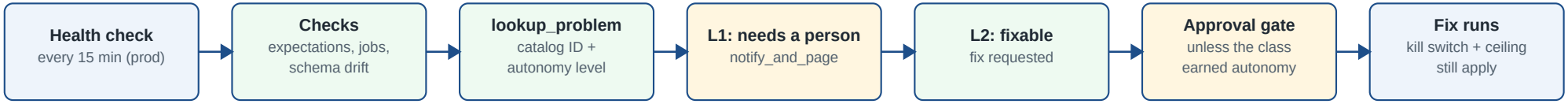
W2. Analyst question

A clinician asks a plain-English question. Routine metrics go to the metric path; anything else to Genie. Our layer decides whether the question may be asked and what of the answer may be shown.

Step	What happens	If it goes wrong	Code	Status
Input check	Requests for patient identifiers refused before anything is sent	Refusal returned; nothing reaches Genie	agent/tools/genie.py	Measured - 3/3 probes refused
Metric path	compute_metric over the metrics layer; same definitions as Genie	Unknown value -> refusal listing allowed values	agent/tools/metrics.py	Measured - 20/20 correct, 1.3 s
Genie	Conversation API: start, poll, fetch result	Failure / timeout -> tool error	agent/databricks_client.py	Measured - 21/21 correct, 13 s median
Scope + consistency	SQL must read only Gold and filter only on values the question named	Result withheld	agent/tools/genie.py	Measured - caught PICU -> ICU
Output guard	Masked columns redacted; COUNT cells under 11 suppressed and Genie's text withheld	Suppressed with a reason	agent/tools/genie.py	Measured - caught a count of 1

W3. Pipeline incident

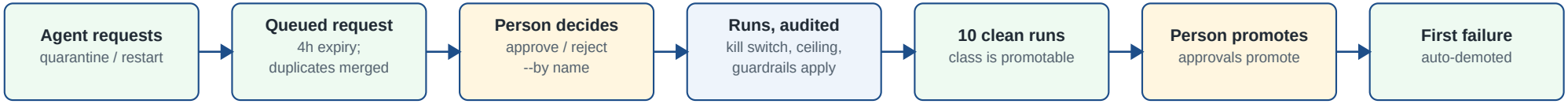
A health check asks the agent to look at the pipelines. The problem catalog decides how far the agent may go.



Step	What happens	Rules	Code	Status
Check	Expectation failures, job states, schema drift	Read-only; each result names its backend	agent/tools/pipeline_health.py, _live.py	Runs live on Free Edition (expectations, job status)
Match	Symptom matched to one of 104 catalog problems	No match = new problem -> escalate	agent/tools/problem_catalog.py	Tested locally
Escalate	Contract breaks, data loss, schema drift go to a person	Never auto-fixed	notify_and_page	Tested locally
Request fix	Quarantine / restart become queued requests	Approval required by default	agent/orchestrator.py	Tested locally
Scheduled job	Serverless job; rules planner when no model key; exit codes 0-3	Non-zero fails the run -> email to the owner	agent/healthcheck.py, agent/rules_planner.py, resources/workflows.yml	Runs - Free Edition: 3 findings, 2 paged, 1 fix queued for approval

W4. Human approval and earned autonomy

How a requested fix becomes an executed one - or doesn't - and how a proven kind of fix stops needing a person.



Command	Effect	Notes	Code	Status
python -m agent.approvals list	Queued, unexpired requests	Read-only	agent/approvals.py	Tested locally
... approve / reject --by "Name"	Executes or closes the request	Both audited with the person's name	agent/approvals.py	Tested locally
... classes	Each action class: policy, streak, eligibility	Streak = consecutive approved, successful runs	agent/autonomy.py	Tested locally
... promote --by "Name"	Class runs without approval from now on	Needs 10 clean runs; quarantine/restart only	agent/autonomy.py	Tested locally
Automatic demotion	First failed automatic run puts the class back on approval	Reason recorded	agent/orchestrator.py	Tested locally

W5. Deploy and run on Databricks Free Edition

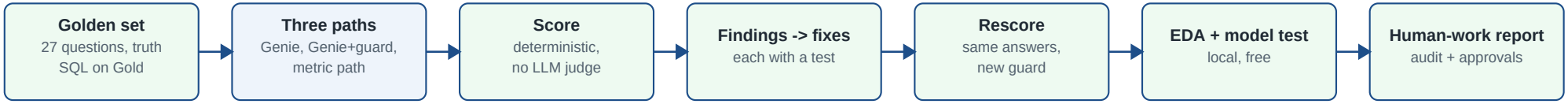
From laptop to a running pipeline on Databricks Free Edition (serverless, no cost, doesn't expire), and back to nothing running. The GCP trial workspace was used first; it locked sign-in when the trial ended.

Step	Command	Cost	Notes	Status
Login	databricks auth login --profile free	Free	OAuth in the browser; the account owner approves	Runs
Catalog, data	CREATE CATALOG / SCHEMA / VOLUME; databricks fs cp	Free	Same catalog name as before, so no code changes	Runs
Deploy + run	databricks bundle deploy -t free; bundle run streaming_patient_pipeline	Free (quota)	Dev mode pauses schedules; triggered	Runs - counts match the local prediction
Evidence	scripts/workspace_snapshot.py	Free	Row counts + event-log expectations into docs/	Runs
Genie + golden set	databricks genie create-space; python -m evals.golden.run	Free (quota)	Run 3: guarded 27/27	Runs
Stop	uv run python scripts/stop_everything.py --profile free	-	Pre-approved after every session	Runs

- The old GCP trial: only the account owner can cancel its Databricks subscription in GCP Marketplace, which guarantees it never bills.

W6. Measure: answers, safety, data, work

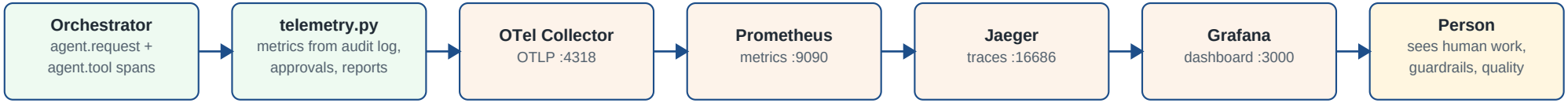
Nothing is claimed without a measurement. Each run's findings become guardrail changes with regression tests.



Measurement	How	Result so far	Code	Status
Golden question set	python -m evals.golden.run (live), .rescore (offline replay)	Run 2 with current guard: 21/21 answers, 6/6 probes; Genie alone failed 1 probe	evals/golden/	Measured
Latency	Same runs	Genie p50 13.4 s, p90 17.1 s; metric path 1.3 s	evals/golden/run.py	Measured
EDA	scripts/eda_landing.py (DuckDB over raw JSON)	Input clean; 13 patient-id collisions; vitals not physiological	docs/eda/	Measured
Detectors	python -m ml.evaluate_anomaly_model	Round 1: model v1 worse than the range rule. Round 2: v2 61% vs NEWS2 47% at equal false alarms	ml/evaluate_anomaly_model.py	Measured - v2 promoted as a calibrated third layer
Human work	python -m agent.human_work	Counts requests, who needed a person, decision times	agent/human_work.py	Tested locally - no real usage yet

W7. Observability

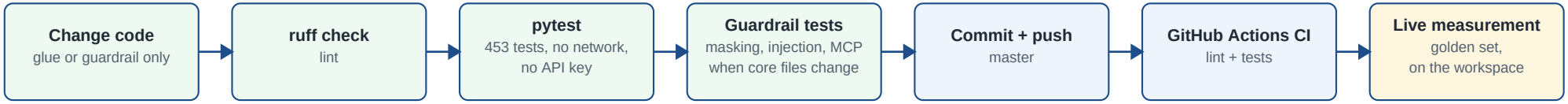
Every request is a trace; the records become metrics; one dashboard shows human work, guardrails and quality. Everything runs locally and free; moving to Datadog, Dynatrace or Grafana Cloud is a collector config change.



Piece	Command	Notes	Code	Status
Start the stack	docker compose --profile observability up -d	Collector, Prometheus, Jaeger, Grafana (pinned images)	docker-compose.yml, observability/	Runs locally
Metrics exporter	python -m agent.telemetry	16 gauges recomputed from the records every 15 s	agent/telemetry.py	Runs locally
Traces	OTEL_EXPORTER_OTLP_ENDPOINT=http://localhost:4318	Spans carry ids, tool names and outcomes - never request text	agent/otel.py, agent/orchestrator.py	Runs locally
Demo traffic	python scripts/demo_traffic.py	Real orchestrator and guardrails, scripted tool choices, no API cost	scripts/demo_traffic.py	Runs locally - demo data, not real usage
Dashboard	python scripts/build_grafana_dashboard.py	Generated; a test fails if a panel queries an unknown metric	observability/grafana/	Runs locally

W8. Development and quality

How a change gets in. The repo's credibility rests on an honest split between runs, measured, tested locally, mock-tested and never run.



Practice	Why	Where	Enforced by	Status
No real secrets in the repo	Public repo	.gitignore, agent/secrets.py	Review; Secret Manager	Tested locally
Status labels in every doc	Never claim untested work works	README feature status, this document	Review	Tested locally
Generated artifacts checked	Genie space and dashboard can't drift from their source	tests/test_metrics_layer.py, test_telemetry.py	CI	Tested locally
Live paths mock-tested	No network in CI	tests/conftest.py clears DATABRICKS_*	conftest fixture	Tested locally
Findings become tests	A fixed failure stays fixed	tests/ (golden, EDA findings)	CI	Tested locally

3. Tech stack

Versions are the ones pinned in uv.lock and docker-compose.yml. Status colours: green runs, measured or tested locally; yellow partial, advisory or mock-tested; red never run; grey planned.

Layer	Technology	Version	What it does here	Repo path	Status
Language / tooling	Python, uv, ruff, pytest	Python >=3.11; ruff 0.16.8; pytest 9.1.1	uv-only package management, lint, 453 tests with zero network calls	pyproject.toml, uv.lock, tests/	Tested locally - passing in CI
Data source	Patient-event simulator (Synthea / MIMIC-IV-shaped, synthetic)	-	Admit / transfer / discharge + 6 vitals; Kafka publisher or JSON-lines files	simulator/	Runs - 103,301 events generated and ingested
Transport (free trial)	Autoloader over a Unity Catalog volume	-	Reads JSON-lines files; no extra service	simulator/file_feed.py, pipeline/01_ingest/	Runs on Databricks
Transport (prod)	Kafka / Redpanda	confluent-kafka 2.15.1	Production event stream; local broker via Docker	docker-compose.yml, simulator/producer.py	Tested locally - not run on Databricks
Pipeline	Lakeflow Declarative Pipelines (formerly DLT), serverless	-	Bronze -> contract gate -> Silver (CDC SCD1 + SCD2) -> Gold	pipeline/, resources/dlt_pipeline.yml	Runs on Databricks - layer counts reconcile
Contract	Pydantic + shared constants	pydantic 2.13.5	Schema, masked columns, vital ranges and physical limits, min cell size	common/contracts.py	Tested locally
Governance	Unity Catalog	-	Column masks and row filters declared in the pipeline; grants	pipeline/04_gold/, governance/05_unity_catalog/	Partial - masks verified (1,388 of 1,388); row filter and grants not tested
Storage / query (live)	Delta tables, serverless SQL warehouse	-	Gold tables; Statement Execution API	agent/databricks_client.py	Runs - golden set queried it live
Storage / query (local)	DuckDB	duckdb 1.5.5	Local Gold stand-in; EDA over the raw JSON lines	data/seed/, scripts/eda_landing.py	Tested locally
Metrics layer	One YAML of metric definitions	pyyaml 6.0.3	Feeds both Genie's instructions and compute_metric	agent/knowledge/metrics.yaml, agent/tools/metrics.py	Runs - live Genie space synced
Platform agent (questions)	Databricks Genie	Conversation API	Writes and runs SQL for analyst questions	agent/tools/genie.py	Measured - golden set, 2 live runs
Platform agent (runbooks)	Databricks Agent Bricks	-	Knowledge agent over the problem catalog	-	Planned
Planner / LLM	Laptop model via Ollama (qwen3:14b), Databricks Model Serving (Qwen 3.5 122B), rules planner; Anthropic SDK optional	Ollama 0.40, Foundation Model APIs	Mode routing and tool loop; no agent framework	agent/local_llm.py, agent/databricks_llm.py, agent/orchestrator.py	Runs - 14 scenario evals: Qwen 3.5 14/14, qwen3:14b on the laptop 12/14
Tool protocol	Model Context Protocol (MCP)	mcp 2.2.0	DE tools served over stdio	mcp_server/, agent/mcp_bridge.py	Tested locally
Knowledge	Problem catalog + lookup_problem	pyyaml 6.0.3	104 real-time problems with expected response and autonomy level	agent/knowledge/problem_catalog.yaml	Tested locally

Layer	Technology	Version	What it does here	Repo path	Status
ML	IsolationForest -> MLflow -> ONNX; NEWS2	scikit-learn 1.9.1, mlflow 3.16.1, onnxruntime 1.30.0	Deterioration: rules, NEWS2, then model v2 at a calibrated threshold	ml/, agent/tools/anomaly_score.py, early_warning.py	Measured - v2 catches 61% vs NEWS2 47% at the same false alarms (simulated)
Simulator physiology	Per-patient baselines, unit acuity, deterioration episodes	-	Realistic, labelled vitals for training and evaluation	simulator/physiology.py	Tested locally - Databricks data predates it
Human approval	Approvals CLI + earned autonomy	-	Named approver, 4h expiry; promotion after 10 clean runs, auto-demotion	agent/approvals.py, agent/autonomy.py	Tested locally
Observability	OpenTelemetry SDK + Collector, Prometheus, Jaeger, Grafana	otel-sdk 1.45.0; collector 0.111.0; Prometheus 2.54.1; Jaeger 1.62.0; Grafana 11.2.2	Request and tool traces; human-work, guardrail and quality metrics; one dashboard	agent/otel.py, agent/telemetry.py, observability/	Runs locally (Docker) - on demo traffic
LLM tracing / logs	Langfuse, structlog	langfuse 4.15.6, structlog 26.1.0	LLM-call traces; structured logs	agent/llm.py	Tested locally (no-op path)
Orchestration / deploy	Databricks Asset Bundles + Workflows (serverless jobs)	Databricks CLI v1.19.0	Pipelines, roster job, health-check job	databricks.yml, resources/	Runs (free target) - health-check job runs serverless
Evaluation	Golden question set; scenario evals	-	27 live questions, 3 configurations; 14 scripted scenarios	evals/golden/, evals/	Measured (golden); scenarios not run on a real model
Secrets	GCP Secret Manager	google-cloud-secret-manager 2.30.0	No tokens in files; no-op unless configured	agent/secrets.py	Tested locally (no-op path)
Infrastructure	Terraform, Cloud Run (GCP)	Terraform >=1.5	GCS bucket, service account, Pub/Sub, Secret Manager	infra/	Never run
CI	GitHub Actions	-	Lint, guardrail tests, tests, pipeline integration, docs in sync; bundle validate/deploy	.github/workflows/	Runs - passing on GitHub
Cost control	stop_everything	-	Cancel runs, pause schedules, stop pipelines and warehouses	scripts/stop_everything.py	Validated - dry run on the workspace

4. Guardrails

Enforced in code on every request, not just asked for in a prompt.

Guardrail	What it stops	Where	Proof
PHI masking before any model	Names / MRNs reaching the LLM, transcript, or audit log	governance_guard, orchestrator _dispatch	tests/test_masking_guard.py
PHI-request refusal	Asking Genie or the fallback for identifiers	agent/tools/genie.py, data_query	tests/test_genie_gateway.py; golden set 3/3 probes
Masking backstop on Genie output	A misconfigured UC mask leaking values, incl. into Genie's text	agent/tools/genie.py	tests/test_genie_gateway.py
SQL scope check	Genie results from tables outside Gold	agent/tools/genie.py	tests/test_genie_gateway.py
Question/SQL consistency	Genie answering a different question (PICU asked, ICU answered)	agent/tools/genie.py	Found by golden run 1; regression test
Small-cell suppression	Counts over fewer than 11 patients, incl. Genie's own text	aggregate_gold_table (exact), ask_genie (COUNT columns)	Found by golden run 2; tests/test_genie_gateway.py
Prompt-injection scan	Instructions hidden in free text	governance_guard.scan_for_injection	tests/test_prompt_injection_guard.py
Rules, then NEWS2, then a calibrated model	A model's miss hiding a dangerous reading (v2 scores SpO2 80 as normal)	agent/tools/anomaly_score.py	ml/evaluate_anomaly_model.py; tests/test_anomaly_score_tool.py
Human approval + earned autonomy	The agent changing pipelines on its own before it has earned it	agent/approvals.py, agent/autonomy.py	tests/test_human_approval.py, test_earned_autonomy.py
Kill switch + escalation ceiling	Remediation loops; fixing past a hard stop	orchestrator	tests/test_kill_switch.py, test_escalation_ceiling.py
Audit log + traces	Untraceable actions; spans never carry request text	orchestrator, agent/otel.py	tests/test_audit_trail.py, test_telemetry.py
Graceful degradation	LLM outage crashing callers or looking green	orchestrator.handle, healthcheck exit 1	tests/test_healthcheck.py

5. Where everything lives

Folder / file	Contents
agent/	The glue-and-guardrail layer: orchestrator, LLM wrapper, approvals, autonomy, human-work report, telemetry, healthcheck, Databricks client, MCP bridge
agent/tools/	genie (Genie gateway), metrics, data_query, pipeline_health (+ _live), problem_catalog, governance_guard, anomaly_score, knowledge_search
agent/knowledge/	metrics.yaml (metrics layer), problem_catalog.yaml (104 problems)
pipeline/, resources/, databricks.yml	Lakeflow pipeline code and the Asset Bundle (dev target on the trial workspace)
governance/	Unity Catalog SQL; genie/space.json (generated Genie space)
evals/	golden/ (live question set, run, rescore, scoring) and scripted scenarios
ml/	Anomaly model training, evaluation, committed ONNX file
observability/	OTel Collector, Prometheus, Grafana provisioning and dashboard
scripts/	eda_landing, demo_traffic, build_* (Genie space, dashboard, images, docs), stop_everything
docs/	plan, architecture, eda/, comparisons/ (Genie results), images/, this document
tests/	453 tests, one file per concern, zero network calls

Pull it

- `gh repo clone sumanthreddy369/databricks-agentic-de`
- `uv sync --extra dev` then `uv run python -m pytest`

6. What's next

#	Step	Needs	Outcome
1	Connect the alert webhook (Slack / Teams) as a Databricks secret; unpause the 15-minute schedule	You: create the webhook and the secret	Alerts in chat, once per problem
2	Unit row filter tested with real workspace groups (a nurse sees only their unit)	Groups in the workspace	Row-level access shown, not just column masks
3	Split shared patient_ids at source (re-assign the second person's admissions) after a steward confirms	Source-system access	Shared ids resolved, not just flagged
4	Grow the runbook question set from real incidents; re-check the 0.55 similarity floor on it	Incident history	Runbook search measured on real questions, not written ones
5	Laptop-only planner by default for anything touching patient data; larger local model if the GPU allows	-	No prompt or tool result leaves the machine
6	Turn on the GitHub -> Databricks stages (validate, deploy on merge, integration run)	You: DATABRICKS_HOST / DATABRICKS_TOKEN repo secrets	Every merge checked against the real workspace
7	Cancel the old GCP trial subscription (that workspace is locked)	You, in GCP Marketplace	No charges